```
%pip install torch
```

```
Defaulting to user installation because normal site-packages is not
writeable
Requirement already satisfied: torch in c:\users\lenovo\appdata\
roaming\python\python313\site-packages (2.9.0)
Requirement already satisfied: filelock in c:\users\lenovo\appdata\
roaming\python\python313\site-packages (from torch) (3.20.0)
Requirement already satisfied: typing-extensions>=4.10.0 in c:\users\
lenovo\appdata\roaming\python\python313\site-packages (from torch)
(4.15.0)
Requirement already satisfied: sympy>=1.13.3 in c:\users\lenovo\
appdata\roaming\python\python313\site-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in c:\users\lenovo\
appdata\roaming\python\python313\site-packages (from torch) (3.5)
Requirement already satisfied: jinja2 in c:\users\lenovo\appdata\
roaming\python\python313\site-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in c:\users\lenovo\
appdata\roaming\python\python313\site-packages (from torch) (2025.9.0)
Requirement already satisfied: setuptools in c:\users\lenovo\appdata\
roaming\python\python313\site-packages (from torch) (80.9.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in c:\users\lenovo\
appdata\roaming\python\python313\site-packages (from sympy>=1.13.3-
>torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\lenovo\
appdata\roaming\python\python313\site-packages (from jinja2->torch)
(3.0.3)
Note: you may need to restart the kernel to use updated packages.
```

```
import torch
import torch.nn as nn

# 1. Data Preparation
text = "I love machine learning and I love deep learning"
words = text.split()

vocab = set(words)
word_to_ix = {w: i for i, w in enumerate(vocab)}
ix_to_word = {i: w for w, i in word_to_ix.items()}

# 2. Generate Training Data
data = []

for i in range(1, len(words) - 1):
    context = [words[i - 1], words[i + 1]]
    target = words[i]
    data.append((context, target))

# 3. Define CBOW Model
```

```python
class CBOW(nn.Module):
    def __init__(self, vocab_size, embedding_dim):
        super().__init__()
        self.embeddings = nn.Embedding(vocab_size, embedding_dim)
        self.linear = nn.Linear(embedding_dim, vocab_size)

    def forward(self, context):
        embeds = self.embeddings(context)
        embeds = embeds.mean(dim=0)
        return self.linear(embeds)

# Initialize model
model = CBOW(len(vocab), 10)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

# 4. Train Model
for epoch in range(100):
    for context, target in data:
        inputs = torch.tensor([word_to_ix[w] for w in context])
        label = torch.tensor([word_to_ix[target]])

        output = model(inputs)
        loss = loss_fn(output.unsqueeze(0), label)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

# 5. Output
context = ["I", "learning"]
inputs = torch.tensor([word_to_ix[w] for w in context])

with torch.no_grad():
    output = model(inputs)
    predicted = ix_to_word[output.argmax().item()]

print("Context words:", context)
print("Predicted word:", predicted)
```

```
Context words: ['I', 'learning']
Predicted word: and
```